

Multi-Agent RAG System Architecture for SeaS Platform

Table of Contents

- [Executive Summary](#)
- [System Overview: Multi-Agent RAG Architecture](#)
- [RAG Agent 1: Report Classification & Verification Agent](#)
- [RAG Agent 2: Resource Allocation & Optimization Agent](#)
- [RAG Agent 3: Proof-of-Action Verification Agent](#)
- [RAG Agent 4: Data Analysis & Insights Agent](#)
- [Cross-Agent Coordination & Integration](#)
- [Implementation Roadmap](#)
- [Conclusion](#)
- [References](#)

Executive Summary

The SeaS (Sustainable Environment As a Service) platform employs a **hierarchical multi-agent RAG architecture** with four specialized agents working collaboratively to transform urban climate resilience. This document defines each RAG agent's specific responsibilities, technical requirements, knowledge bases, and integration patterns aligned with SDG 13 objectives.

System Overview: Multi-Agent RAG Architecture

The SeaS platform implements an **orchestrator-worker pattern** where four specialized RAG agents operate in coordinated workflows [1][44][64][69]:

- **Agent 1: Report Classification & Verification Agent** (Authenticity Guardian)
- **Agent 2: Resource Allocation & Optimization Agent** (Smart Allocator)
- **Agent 3: Proof-of-Action Verification Agent** (Accountability Enforcer)
- **Agent 4: Data Analysis & Insights Agent** (Intelligence Synthesizer)

Each agent maintains its own domain-specific knowledge base while sharing a unified vector store infrastructure for cross-modal information retrieval [1][52][64].

RAG Agent 1: Report Classification & Verification Agent

Primary Responsibilities

This agent serves as the **first line of defense** in the complaint pipeline, ensuring that only legitimate, accurately categorized issues enter the system [1][45][48].

Core Functions:

1. **Automated Issue Classification:** Categorize complaints into predefined types (drainage, waste dumping, illegal construction, green cover loss, air quality concerns)
2. **Fraud Detection & Authenticity Verification:** Identify suspicious patterns, duplicate submissions, AI-generated fake images, and fraudulent complaints
3. **Climate Risk Prioritization:** Apply SDG 13 climate adaptation checks to assign urgency scores based on vulnerability assessments
4. **Multilingual Natural Language Understanding:** Process complaints in Marathi, Hindi, and English with contextual comprehension

Technical Architecture

RAG Pipeline Components:

```
User Complaint → Multimodal Embedding → Vector Similarity Search →  
LLM Classification → Fraud Pattern Matching → Climate Risk Scoring →  
Verified Output + Explanation
```

Retrieval Strategy:

- **Hybrid Search:** Combines semantic search (vector similarity) with keyword-based filtering for precise classification [1][53][65]
- **Metadata Filtering:** GPS location, timestamp, user history, device fingerprint
- **Cross-modal Retrieval:** Text descriptions matched with image embeddings using CLIP-style models [1][52]

Generation Strategy:

- **Chain-of-Thought Reasoning:** LLM provides step-by-step explanation for classification decision [1][45]
- **Confidence Scoring:** Assigns probability scores to each classification category
- **Human-in-the-Loop Flagging:** Low-confidence cases (< 70%) routed to manual review [1]

Knowledge Base Requirements

KB-1A: Historical Complaint Database

- **Content:** 500K+ verified past complaints with labels, resolutions, and outcomes
- **Structure:** Vector embeddings of complaint text + image features + metadata (location, time, category, resolution time)
- **Update Frequency:** Real-time ingestion as new complaints are resolved
- **Embedding Model:** Multilingual sentence transformer (e.g., paraphrase-multilingual-mpnet-base-v2) [1][53]
- **Vector Dimension:** 768-dimensional embeddings
- **Storage:** PostgreSQL with pgvector extension or Pinecone [1][65][68]

KB-1B: Fraud Pattern Library

- **Content:** Known fraud patterns including:
 - Duplicate image hashes and perceptual hashes
 - GPS spoofing signatures (impossible travel speeds, location clustering)
 - AI-generated image artifacts (GAN fingerprints, DALL-E watermarks)
 - Mass complaint campaigns (coordinated timing, similar text patterns)
 - Fake account indicators (burner phone numbers, disposable emails)
- **Structure:** Rule-based patterns + anomaly detection models + adversarial examples
- **Update Frequency:** Weekly updates from security team + automated anomaly detection
- **Storage:** Redis cache for fast pattern matching + Neo4j graph database for relationship tracking [1][45][48]

KB-1C: Municipal Infrastructure Asset Registry

- **Content:** Geolocated database of all civic assets (drains, waste bins, street lights, parks, water bodies)
- **Structure:** GeoJSON with attribute data (asset ID, type, capacity, maintenance history, condition score)
- **Purpose:** Verify complaint locations correspond to real infrastructure
- **Update Frequency:** Monthly sync with municipal GIS systems
- **Storage:** PostGIS spatial database [1]

KB-1D: Climate Vulnerability Index Database

- **Content:** Ward-level climate risk scores based on:
 - Flood risk zones (historical flooding data, drainage capacity)
 - Heat island intensity (NDVI data, built-up area percentage)

- Air quality trends (PM2.5, NOx levels)
- Population density and vulnerable demographics
- **Structure:** Geospatial risk heatmaps with temporal dimensions
- **Purpose:** Prioritize complaints from high-vulnerability areas for SDG 13 alignment [1][2]
- **Update Frequency:** Quarterly updates with satellite data from ISRO Bhuvan
- **Storage:** InfluxDB for time-series climate data + PostGIS for spatial layers [1]

KB-1E: Policy & Regulation Knowledge Base

- **Content:**
 - Municipal bylaws and environmental regulations
 - SDG 13 indicators and national climate policies
 - Penalty structures for violations
 - Escalation procedures for critical issues
- **Structure:** Hierarchical document chunks with legal references
- **Purpose:** Ensure compliance checks and appropriate routing
- **Update Frequency:** As regulations change (event-driven updates)
- **Storage:** Elasticsearch for document retrieval [1][44]

Functional Requirements

FR-RAG1-01: The agent SHALL classify incoming complaints into 12+ predefined categories with minimum 90% accuracy on test dataset [1][26].

FR-RAG1-02: The agent SHALL process multimodal inputs (text + images + GPS + audio) and generate unified semantic embeddings [1][52][61].

FR-RAG1-03: The agent SHALL detect AI-generated images using adversarial detection models with minimum 95% precision [1][45].

FR-RAG1-04: The agent SHALL identify duplicate complaints by comparing perceptual image hashes and text similarity scores (threshold: 85% similarity) [1][45].

FR-RAG1-05: The agent SHALL flag GPS spoofing attempts by validating:

- Location corresponds to real infrastructure asset ($\pm 50\text{m}$ radius)
- Travel speed between consecutive complaints is humanly possible ($< 120\text{ km/h}$)
- Device location history consistency [1][45]

FR-RAG1-06: The agent SHALL assign climate vulnerability scores (1-10 scale) based on ward-level risk indices and complaint type [1][2].

FR-RAG1-07: The agent SHALL provide explainable AI outputs including:

- Top 3 similar historical cases with resolution times
- Classification confidence score
- Fraud risk assessment with evidence
- Climate priority justification [1][45][48]

FR-RAG1-08: The agent SHALL support multilingual processing with automatic language detection and translation to English for internal processing [1].

FR-RAG1-09: The agent SHALL maintain audit logs of all classification decisions with timestamp, input data hash, and model version [1][10].

FR-RAG1-10: The agent SHALL route low-confidence classifications ($< 70\%$) to human verifiers within 30 seconds [1].

Non-Functional Requirements

- NFR-RAG1-01: Response Time:** Complete classification within 3 seconds for 95th percentile requests [4][10].
- NFR-RAG1-02: Throughput:** Handle 1,000 concurrent complaint submissions without performance degradation [10][13].
- NFR-RAG1-03: Accuracy:** Maintain minimum 92% F1-score on fraud detection across 6-month evaluation period [1][45].
- NFR-RAG1-04: Availability:** 99.5% uptime with automatic failover to backup LLM provider [10].
- NFR-RAG1-05: Explainability:** Generate human-readable explanations for 100% of classification decisions [1][48].
- NFR-RAG1-06: Multilingual Support:** Support 3 languages (English, Hindi, Marathi) with < 5% accuracy degradation compared to English-only [1].
- NFR-RAG1-07: Model Drift Detection:** Automatically trigger retraining when classification accuracy drops below 88% over 2-week window [1].

Technology Stack

- **LLM:** GPT-4o / Claude 3.5 Sonnet / Groq Llama 3.1 70B (for cost efficiency) [1]
- **Embedding Model:** paraphrase-multilingual-mpnet-base-v2 (768-dim) for text, CLIP ViT-L/14 for images [1][52][53]
- **Vector Database:** Pinecone (managed, scalable) or PostgreSQL with pgvector [1][65][68]
- **Fraud Detection:** Isolation Forest for anomaly detection + custom CNN for deepfake detection [1][45]
- **Orchestration:** LangGraph for agent workflow management [1][64]
- **API Framework:** FastAPI for REST endpoints [1]

Input/Output Specifications

Input Schema:

```
{
  "complaint_id": "string (UUID)",
  "user_id": "string",
  "timestamp": "ISO 8601 datetime",
  "location": {
    "latitude": "float",
    "longitude": "float",
    "accuracy": "float (meters)"
  },
  "text_description": "string (max 500 chars)",
  "language": "string (en/hi/mr)",
  "images": ["base64 encoded images"],
  "audio": "base64 encoded (optional)",
  "device_metadata": {
    "device_id": "string",
    "os": "string",
    "app_version": "string"
  }
}
```

Output Schema:

```
{
  "complaint_id": "string",
  "classification": {
    "category": "string",
    "subcategory": "string",
    "confidence_score": "float (0-1)",
    "similar_cases": [
      {
        "case_id": "string",
        "similarity_score": "float",
        "resolution_time_days": "int"
      }
    ]
  }
}
```

```

    ]
  },
  "authenticity_check": {
    "is_authentic": "boolean",
    "fraud_risk_score": "float (0-1)",
    "fraud_indicators": ["array of strings"],
    "image_authenticity": "boolean",
    "gps_validation": "boolean"
  },
  "climate_priority": {
    "vulnerability_score": "int (1-10)",
    "ward_risk_level": "string (low/medium/high/critical)",
    "sdg13_alignment": "boolean",
    "priority_rank": "int"
  },
  "explanation": "string (human-readable reasoning)",
  "recommended_action": "string",
  "requires_human_review": "boolean",
  "processing_time_ms": "int"
}

```

RAG Agent 2: Resource Allocation & Optimization Agent

Primary Responsibilities

This agent acts as the **intelligent dispatcher**, optimizing the allocation of municipal resources (vehicles, personnel, equipment) to maximize efficiency, minimize costs, and reduce environmental impact [1][63][72].

Core Functions:

1. **Dynamic Task Assignment:** Match verified complaints to appropriate field workers and vehicles
2. **Route Optimization:** Generate fuel-efficient collection routes using real-time traffic and facility capacity
3. **Capacity Planning:** Predict resource needs based on historical patterns and forecasted demand
4. **Sustainability Metrics Calculation:** Compute CO2 savings, fuel reduction, and environmental impact

Technical Architecture

RAG Pipeline Components:

```

Verified Complaints + Resource State → Retrieval of Historical Patterns →
LLM-Driven Optimization → Route Generation →
Sustainability Impact Calculation → Assignment + Explanation

```

Retrieval Strategy:

- **Time-Series Similarity Search:** Find similar demand patterns from historical data [1][63]
- **Constraint-Based Filtering:** Filter by vehicle capacity, fuel level, worker availability, facility operating hours
- **Graph-Based Retrieval:** Traverse road network graph for route optimization [1][65]

Generation Strategy:

- **Multi-Objective Optimization:** LLM balances competing objectives (speed, cost, emissions, fairness) [1][63][72]
- **Policy-Guided Allocation:** Respects business rules (worker shift limits, vehicle maintenance schedules) [1][63]
- **Reinforcement Learning Integration:** Continuously improve allocation strategies based on outcome feedback [1][63][72]

Knowledge Base Requirements

KB-2A: Fleet & Personnel Database

- **Content:** Real-time inventory of all resources:
 - Vehicles: Type, capacity, fuel level, current location, availability status, maintenance schedule
 - Field workers: Skills, shift schedule, current workload, performance ratings, languages spoken
 - Equipment: Availability, condition, last service date
- **Structure:** Real-time state database with temporal history
- **Update Frequency:** Real-time updates via IoT sensors and mobile app
- **Storage:** Redis for current state + PostgreSQL for historical records [1][63]

KB-2B: Historical Assignment Patterns

- **Content:** 2+ years of assignment data including:
 - Task characteristics → Assigned resources
 - Completion times, success rates, citizen satisfaction scores
 - Seasonal demand variations
 - Worker performance by complaint type
- **Structure:** Time-series embeddings capturing temporal patterns
- **Purpose:** Learn optimal assignment strategies through pattern matching
- **Embedding Model:** Temporal LSTM-based embeddings [1]
- **Storage:** InfluxDB for time-series + vector database for semantic search [1][53]

KB-2C: Road Network & Traffic Database

- **Content:**
 - OpenStreetMap road network for city
 - Real-time traffic data from Google Maps API / municipal traffic sensors
 - Historical traffic patterns by time-of-day and day-of-week
 - Road closures, construction zones, restricted areas
- **Structure:** Directed graph with edge weights representing travel time
- **Purpose:** Calculate optimal routes considering real-time conditions
- **Update Frequency:** Traffic data updates every 5 minutes
- **Storage:** Neo4j graph database for route queries [1][65]

KB-2D: Waste Facility Registry

- **Content:** All processing/disposal facilities with:
 - Location, operating hours, accepted waste types
 - Current capacity utilization (real-time)
 - Processing rates, tipping fees
 - Environmental compliance records
- **Structure:** GeoJSON with real-time capacity attributes
- **Purpose:** Match waste pickups to nearest suitable facility with available capacity [1]
- **Update Frequency:** Real-time capacity updates from facility management systems
- **Storage:** PostGIS + Redis cache [1]

KB-2E: Optimization Policy Library

- **Content:** Business rules and constraints:
 - Worker shift regulations (max 8 hours, mandatory breaks)
 - Vehicle fuel efficiency curves

- Priority rules (emergency complaints first, climate-vulnerable areas prioritized)
- Union agreements, labor laws
- Budget constraints
- **Structure:** Rule engine with priority hierarchy
- **Purpose:** Ensure allocations comply with regulations and policies [1][63]
- **Update Frequency:** As policies change (manual updates)
- **Storage:** YAML configuration files + validation engine [1]

KB-2F: Sustainability Impact Models

- **Content:** Calculation formulas and emission factors:
 - Fuel consumption models by vehicle type and speed
 - CO₂e emission factors (2.68 kg CO₂e/L diesel) [1]
 - Alternative route environmental impact comparisons
 - Waste diversion impact calculations
- **Structure:** Mathematical models + lookup tables
- **Purpose:** Compute environmental metrics for SDG 13 reporting [1][2]
- **Storage:** Python calculation library + PostgreSQL for factor tables [1]

Functional Requirements

FR-RAG2-01: The agent SHALL generate optimal resource assignments within 5 seconds of receiving verified complaint batches [1][63].

FR-RAG2-02: The agent SHALL optimize routes to minimize total distance traveled while respecting:

- Vehicle capacity constraints
- Facility operating hours
- Worker shift limits
- Traffic conditions [1][63][72]

FR-RAG2-03: The agent SHALL prioritize complaints based on:

- Climate vulnerability score (from Agent 1)
- Complaint age (older complaints prioritized)
- Citizen-reported urgency
- SDG 13 climate adaptation priorities [1][2][63]

FR-RAG2-04: The agent SHALL calculate and display sustainability metrics for each route:

- Fuel saved (L) = (Baseline Distance - Optimized Distance) × Fuel Rate
- CO₂e reduced (kg) = Fuel Saved × 2.68 kg/L
- Cost savings (₹) = Fuel Saved × Price per L [1]

FR-RAG2-05: The agent SHALL balance workload across field workers to prevent burnout (max 15% deviation from average) [1][63].

FR-RAG2-06: The agent SHALL detect and flag impossible assignments (e.g., vehicle at 10% fuel cannot complete 50km route) [1][63].

FR-RAG2-07: The agent SHALL support dynamic re-optimization when:

- New urgent complaints arrive
- Vehicle breakdowns occur
- Traffic conditions change significantly
- Weather emergencies declared [1][63][72]

FR-RAG2-08: The agent SHALL learn from historical outcomes using reinforcement learning:

- Actual completion times vs. estimates
- Citizen satisfaction scores
- Resource utilization efficiency [1][63][72]

FR-RAG2-09: The agent SHALL generate human-readable assignment explanations:

- Why specific worker assigned to task
- Why specific route chosen
- Expected sustainability impact [1][63]

FR-RAG2-10: The agent SHALL maintain fairness by ensuring:

- All workers receive equitable task distribution
- High-priority areas don't monopolize resources
- Suburban areas receive adequate service [1][63]

Non-Functional Requirements

NFR-RAG2-01: Optimization Time: Generate optimal assignments for 100 complaints in < 10 seconds [1][63].

NFR-RAG2-02: Route Efficiency: Achieve minimum 25% reduction in travel distance compared to unoptimized baseline [1].

NFR-RAG2-03: Scalability: Support scaling to 10,000+ daily complaints across 100+ wards [1][63].

NFR-RAG2-04: Real-Time Adaptation: Incorporate real-time changes (traffic, breakdowns) within 2 minutes [1][63][72].

NFR-RAG2-05: Accuracy: Route time estimates within $\pm 15\%$ of actual completion times for 85% of routes [1][63].

NFR-RAG2-06: Sustainability Impact: Achieve minimum 15% CO2e reduction compared to unoptimized operations [1][2].

Technology Stack

- **LLM:** GPT-4o for complex multi-objective reasoning / Groq for speed [1]
- **Optimization:** OR-Tools (Google) for constraint programming + custom RL models [1][63]
- **Graph Database:** Neo4j for road network and route optimization [1][65]
- **Time-Series DB:** InfluxDB for traffic patterns and demand forecasting [1]
- **Reinforcement Learning:** Stable-Baselines3 with PPO algorithm [1][63][72]
- **Routing API:** OSRM (Open Source Routing Machine) for route calculations [1]
- **Orchestration:** LangGraph for agent workflows [1][64]

Input/Output Specifications

Input Schema:

```
{
  "batch_id": "string",
  "timestamp": "ISO 8601",
  "complaints": [
    {
      "complaint_id": "string",
      "location": {"lat": "float", "lon": "float"},
      "category": "string",
      "priority_score": "int (1-10)",
      "estimated_workload_minutes": "int"
    }
  ],
  "available_resources": {
    "vehicles": [
      {
        "vehicle_id": "string",
        "type": "string",
        "capacity_kg": "float",
```



```

        "fuel_level_percent": "float",
        "current_location": {"lat": "float", "lon": "float"},
        "available": "boolean"
    }
],
"workers": [
    {
        "worker_id": "string",
        "skills": ["array of strings"],
        "shift_remaining_hours": "float",
        "current_location": {"lat": "float", "lon": "float"},
        "current_workload": "int"
    }
]
},
"facilities": [
    {
        "facility_id": "string",
        "location": {"lat": "float", "lon": "float"},
        "accepted_waste_types": ["array"],
        "current_capacity_percent": "float",
        "operating_hours": "string"
    }
]
}

```

Output Schema:

```

{
    "batch_id": "string",
    "assignments": [
        {
            "complaint_id": "string",
            "assigned_worker": "string",
            "assigned_vehicle": "string",
            "assigned_facility": "string",
            "estimated_start_time": "ISO 8601",
            "estimated_completion_time": "ISO 8601",
            "assignment_reason": "string"
        }
    ],
    "optimized_routes": [
        {
            "route_id": "string",
            "vehicle_id": "string",
            "waypoints": [
                {"complaint_id": "string", "location": {"lat": "float", "lon": "float"}, "sequence": "int"}
            ],
            "total_distance_km": "float",
            "estimated_duration_minutes": "int",
            "fuel_consumption_liters": "float"
        }
    ],
    "sustainability_impact": {
        "baseline_distance_km": "float",
        "optimized_distance_km": "float",
        "distance_saved_km": "float",
        "fuel_saved_liters": "float",
        "co2e_reduced_kg": "float",
        "cost_savings_inr": "float"
    },
    "workload_distribution": {
        "worker_assignments": [
            {"worker_id": "string", "task_count": "int", "estimated_hours": "float"}
        ],
        "fairness_score": "float (0-1)"
    },
    "optimization_metadata": {
        "computation_time_ms": "int",
        "algorithm_used": "string",
        "confidence_score": "float"
    }
}

```

```
}  
}
```

RAG Agent 3: Proof-of-Action Verification Agent

Primary Responsibilities

This agent ensures **accountability and transparency** by verifying that field workers have genuinely completed assigned tasks through photographic evidence and location validation [1].

Core Functions:

1. **Before-After Image Comparison:** Analyze uploaded proof photos to verify work completion
2. **Location Verification:** Validate GPS coordinates match complaint location
3. **Quality Assessment:** Evaluate work quality based on visual evidence
4. **Anomaly Detection:** Identify fake proofs (recycled photos, photoshopped images, wrong locations)

Technical Architecture

RAG Pipeline Components:

```
Proof Submission (images + GPS + timestamp) → Multimodal Embedding →  
Similarity Search (complaint images vs proof images) →  
LLM Visual Reasoning → Location Validation →  
Quality Scoring → Verification Decision + Explanation
```

Retrieval Strategy:

- **Visual Similarity Search:** Compare proof images against original complaint images [1][52]
- **Geospatial Proximity Search:** Verify location within acceptable radius of complaint location
- **Temporal Consistency Check:** Retrieve worker's movement history to validate timeline [1]

Generation Strategy:

- **Vision-Language Models:** GPT-4 Vision / Claude 3.5 Sonnet analyze before-after images and generate natural language assessments [1][52]
- **Structured Output:** Extract specific work indicators (drain unclogged, waste collected, repairs visible) [1]

Knowledge Base Requirements

KB-3A: Original Complaint Evidence Repository

- **Content:** All original complaint submissions with:
 - Photos, videos, audio descriptions
 - GPS coordinates with accuracy
 - Timestamp
 - Environmental context (weather, lighting conditions)
- **Structure:** Multimodal embeddings linking complaint_id to visual/spatial features
- **Purpose:** Provide baseline "before" state for comparison [1]
- **Storage:** S3/Cloud Storage for raw media + vector DB for embeddings [1][52]

KB-3B: Verified Resolution Reference Library

- **Content:** 100K+ examples of verified before-after pairs showing successful work completion:
 - Unclogged drains (before: water stagnation, after: clear flow)
 - Waste removal (before: piles of garbage, after: clean area)

- Repairs (before: pothole, after: patched surface)
- **Structure:** Labeled image pairs with quality scores
- **Purpose:** Train verification models and provide few-shot examples to LLM [1][52]
- **Embedding Model:** CLIP ViT-L/14 for image embeddings [1][52][53]
- **Storage:** Vector database (Pinecone/Qdrant) [1][68]

KB-3C: Fake Proof Detection Database

- **Content:** Known fraud patterns:
 - Recycled photos (image hashes of previously submitted proofs)
 - Stock photos from web (reverse image search results)
 - Photoshopped evidence (manipulation artifacts)
 - Wrong location submissions (GPS doesn't match complaint)
- **Structure:** Blacklist of image hashes + anomaly detection models
- **Purpose:** Detect fraudulent proof submissions [1][45][48]
- **Update Frequency:** Weekly updates + real-time flagging from human reviewers
- **Storage:** Redis for hash lookup + anomaly detection models [1][45]

KB-3D: Quality Standards & Acceptance Criteria

- **Content:** Visual criteria for acceptable work completion by issue type:
 - Drainage: Water flow restored, no visible blockages
 - Waste collection: Area clear of litter, bins emptied
 - Infrastructure repair: Damage fixed, area safe for public use
- **Structure:** Decision trees + visual examples + rubrics [1]
- **Purpose:** Standardize quality assessments across complaints [1]
- **Storage:** Document database (MongoDB) [1]

KB-3E: Worker Movement & Activity History

- **Content:** GPS traces of field workers during shift:
 - Location pings every 30 seconds
 - Activity timestamps (arrived at location, started work, completed work)
 - Historical patterns (typical time spent per task type)
- **Structure:** Time-series geospatial data
- **Purpose:** Validate worker was physically present at complaint location during claimed work time [1]
- **Storage:** InfluxDB for time-series GPS data [1]

Functional Requirements

FR-RAG3-01: The agent SHALL compare proof images against original complaint images using visual similarity models [1][52].

FR-RAG3-02: The agent SHALL validate GPS coordinates are within 50 meters of original complaint location [1].

FR-RAG3-03: The agent SHALL detect recycled photos by comparing image hashes against historical proof submissions (perceptual hashing with 90% similarity threshold) [1][45].

FR-RAG3-04: The agent SHALL identify photoshopped images using manipulation detection algorithms (minimum 85% detection rate) [1][45].

FR-RAG3-05: The agent SHALL verify timestamp consistency:

- Proof timestamp is after complaint timestamp
- Proof timestamp is within worker's shift hours
- Time interval between assignment and completion is reasonable for task type [1]

FR-RAG3-06: The agent SHALL assess work quality using vision-language models to:

- Identify visible improvements in proof images
- Detect incomplete work
- Flag safety hazards or code violations [1][52]

FR-RAG3-07: The agent SHALL assign verification status:

- **VERIFIED:** Clear evidence of work completion, all checks passed
- **QUESTIONABLE:** Minor discrepancies, requires human review
- **REJECTED:** Failed critical checks (wrong location, recycled photo, no visible work) [1]

FR-RAG3-08: The agent SHALL generate structured verification reports including:

- Before-after image comparison summary
- Location validation result
- Quality assessment score (1-10)
- Specific issues detected
- Recommendation (approve/reject/review) [1][52]

FR-RAG3-09: The agent SHALL flag workers with repeated verification failures (3+ rejections in 30 days) for performance review [1].

FR-RAG3-10: The agent SHALL support citizen feedback integration:

- Cross-reference agent verification with citizen satisfaction ratings
- Learn from discrepancies to improve models [1]

Non-Functional Requirements

NFR-RAG3-01: Verification Speed: Complete verification within 5 seconds for 90th percentile submissions [1][4].

NFR-RAG3-02: Accuracy: Achieve minimum 93% agreement with human expert verifiers on test dataset [1][52].

NFR-RAG3-03: Fraud Detection Rate: Detect 95%+ of fake proofs while maintaining false positive rate < 5% [1][45].

NFR-RAG3-04: Explainability: Provide visual highlights and textual explanations for all verification decisions [1][48][52].

NFR-RAG3-05: Multimodal Support: Process images, videos, and audio recordings as proof formats [1][52].

NFR-RAG3-06: Scalability: Handle 10,000 proof submissions per hour during peak times [1][10].

Technology Stack

- **Vision-Language Model:** GPT-4 Vision / Claude 3.5 Sonnet / LLaVA (open-source) [1][52]
- **Image Embeddings:** CLIP ViT-L/14 for semantic image understanding [1][52][53]
- **Manipulation Detection:** CNN-based deepfake detector + ELA (Error Level Analysis) [1][45]
- **Perceptual Hashing:** pHash algorithm for duplicate detection [1][45]
- **Geospatial Validation:** PostGIS spatial queries [1]
- **Orchestration:** LangGraph for workflow [1][64]

Input/Output Specifications

Input Schema:

```
{
  "proof_id": "string (UUID)",
  "complaint_id": "string",
  "worker_id": "string",
  "submission_timestamp": "ISO 8601",
  "location": {
    "latitude": "float",
```

```

    "longitude": "float",
    "accuracy": "float (meters)"
  },
  "proof_images": ["array of base64 encoded images"],
  "proof_video": "base64 encoded (optional)",
  "worker_notes": "string",
  "disposal_receipt": "base64 encoded image (for waste collection)"
}

```

Output Schema:

```

{
  "proof_id": "string",
  "complaint_id": "string",
  "verification_status": "string (VERIFIED/QUESTIONABLE/REJECTED)",
  "verification_timestamp": "ISO 8601",
  "location_validation": {
    "distance_from_complaint_meters": "float",
    "within_acceptable_radius": "boolean",
    "validation_passed": "boolean"
  },
  "image_analysis": {
    "before_after_comparison": {
      "visual_change_detected": "boolean",
      "change_description": "string",
      "improvement_visible": "boolean"
    },
    "quality_assessment": {
      "work_completion_score": "int (1-10)",
      "issues_detected": ["array of strings"],
      "meets_standards": "boolean"
    },
    "authenticity_check": {
      "is_original_photo": "boolean",
      "recycled_photo_detected": "boolean",
      "manipulation_detected": "boolean",
      "fraud_risk_score": "float (0-1)"
    }
  },
  "timeline_validation": {
    "worker_at_location": "boolean",
    "time_spent_minutes": "int",
    "reasonable_duration": "boolean"
  },
  "explanation": "string (detailed reasoning for verification decision)",
  "recommendation": "string (approve/reject/human_review)",
  "flagged_for_review": "boolean",
  "processing_time_ms": "int"
}

```

RAG Agent 4: Data Analysis & Insights Agent

Primary Responsibilities

This agent serves as the **intelligence layer**, transforming raw operational data into actionable insights for municipal officials, researchers, and policymakers [1][54][83].

Core Functions:

1. **Trend Analysis:** Identify patterns in complaint data, resolution times, and environmental metrics
2. **Predictive Analytics:** Forecast future demand, resource needs, and climate risks
3. **Performance Monitoring:** Track ward-level KPIs and generate comparative benchmarks
4. **Report Generation:** Create SDG 13-aligned reports and public dashboards

Technical Architecture

RAG Pipeline Components:

```
User Query → Query Understanding → Retrieval of Relevant Data Subsets →  
Statistical Analysis → LLM Synthesis →  
Visualization Generation → Report + Insights + Recommendations
```

Retrieval Strategy:

- **Time-Series Retrieval:** Query historical data by time ranges, wards, categories [1][53]
- **Aggregated Statistics Retrieval:** Fetch pre-computed KPIs and summaries [1]
- **Cross-Dataset Joins:** Combine complaint data + climate data + demographic data for multi-dimensional analysis [1][83]

Generation Strategy:

- **Data-to-Text:** LLM generates natural language explanations of statistical findings [1][54]
- **Insight Extraction:** Identify anomalies, trends, and correlations automatically [1][54]
- **Recommendation Engine:** Suggest policy interventions based on data patterns [1][54]

Knowledge Base Requirements

KB-4A: Comprehensive Operational Data Warehouse

- **Content:** Complete historical records including:
 - All complaints (original, classified, verified, resolved)
 - Resource allocations and route optimizations
 - Proof verifications and quality assessments
 - Citizen feedback and satisfaction scores
 - Sustainability metrics (CO2 saved, fuel consumed, costs)
- **Structure:** Star schema data warehouse with fact tables (complaints, assignments, verifications) and dimension tables (time, location, categories, workers)
- **Purpose:** Central repository for all analytical queries [1][86]
- **Update Frequency:** Real-time streaming ingestion + nightly aggregations
- **Storage:** PostgreSQL data warehouse + Apache Kafka for streaming [1]

KB-4B: Environmental & Climate Data Repository

- **Content:** Multi-source environmental datasets:
 - AQI data (PM2.5, PM10, NOx, O3) from CPCB sensors
 - Weather data (temperature, rainfall, humidity) from IMD
 - Satellite imagery (NDVI, land surface temperature) from ISRO Bhuvan
 - Groundwater levels from state monitoring wells
 - Green cover percentage by ward
- **Structure:** Time-series + geospatial layers
- **Purpose:** Correlate environmental conditions with complaint patterns [1][2]
- **Update Frequency:** Daily for AQI/weather, monthly for satellite data
- **Storage:** InfluxDB for time-series + PostGIS for spatial data [1]

KB-4C: Demographic & Socioeconomic Database

- **Content:** Census and survey data:
 - Population density, age distribution, literacy rates
 - Income levels, employment statistics
 - Housing conditions, slum areas

- Vulnerable populations (elderly, disabled, low-income)
- **Structure:** Ward-level aggregated statistics
- **Purpose:** Understand social determinants of environmental issues and service equity [1][83]
- **Storage:** PostgreSQL [1]

KB-4D: Policy & Best Practices Library

- **Content:** External knowledge sources:
 - International case studies (successful waste management programs)
 - Academic research papers on urban climate resilience
 - SDG 13 implementation guidelines
 - Municipal best practices from other cities
- **Structure:** Document embeddings for semantic search [1][53][86]
- **Purpose:** Provide context and recommendations based on global knowledge [1][54]
- **Embedding Model:** sentence-transformers/all-MiniLM-L6-v2 [1][53]
- **Storage:** Elasticsearch for document retrieval [1][86]

KB-4E: Pre-Computed Analytics & KPI Cache

- **Content:** Frequently accessed metrics and dashboards:
 - Ward-level KPIs (avg resolution time, complaint volume, recycling %, green cover %)
 - City-wide aggregates
 - Trend lines and moving averages
 - Performance rankings (ward leaderboards)
- **Structure:** Materialized views and cached query results
- **Purpose:** Accelerate dashboard loading and common queries [1]
- **Update Frequency:** Hourly recalculation
- **Storage:** Redis cache + PostgreSQL materialized views [1]

KB-4F: ML Model Repository

- **Content:** Trained predictive models:
 - XGBoost: Complaint volume forecasting (next 7 days)
 - LSTM: AQI prediction (72-hour forecast)
 - Random Forest: Resource demand prediction
 - Clustering: Identify complaint hotspots
- **Structure:** Serialized model artifacts + metadata (version, accuracy, training date)
- **Purpose:** Power predictive analytics features [1]
- **Storage:** MLflow model registry [1]

Functional Requirements

FR-RAG4-01: The agent SHALL support natural language queries such as:

- "What were the top 5 complaint categories in Kothrud ward last month?"
- "Show me the trend in average resolution time over the past year"
- "Which wards have the worst air quality and highest complaint volumes?"
- "Predict waste collection demand for next week" [1][54][83]

FR-RAG4-02: The agent SHALL retrieve relevant data subsets by:

- Time range filtering (last 7 days, last month, last year, custom range)
- Spatial filtering (ward, zone, city-wide)

- Category filtering (drainage, waste, air quality)
- Demographic filtering (high-income areas, slum areas) [1][83]

FR-RAG4-03: The agent SHALL perform statistical analyses including:

- Descriptive statistics (mean, median, percentiles, distribution)
- Correlation analysis (relationship between AQI and health complaints)
- Trend detection (increasing/decreasing patterns)
- Anomaly detection (unusual spikes in complaints) [1][54]

FR-RAG4-04: The agent SHALL generate predictive forecasts:

- Complaint volume for next 7 days (95% confidence intervals)
- AQI levels for next 72 hours
- Resource requirements for upcoming week
- Climate vulnerability risk alerts [1][2][54]

FR-RAG4-05: The agent SHALL calculate ward-level KPIs aligned with SDG 13:

- Average complaint resolution time
- Complaint volume per 10,000 residents
- Waste diversion rate (% diverted from landfills)
- Recycling percentage
- Green cover change (% increase/decrease)
- CO2e emissions reduced (tons/year)
- Citizen satisfaction score (1-5 scale) [1][2][5]

FR-RAG4-06: The agent SHALL generate comparative benchmarks:

- Rank wards by performance metrics
- Compare current performance to historical baselines
- Identify best-performing and worst-performing wards
- Generate city-wide leaderboards [1]

FR-RAG4-07: The agent SHALL create visualizations:

- Time-series line charts (complaint trends)
- Geospatial heatmaps (complaint density)
- Bar charts (category distributions)
- Scatter plots (correlations)
- Tables (KPI summaries) [1]

FR-RAG4-08: The agent SHALL generate automated reports:

- Weekly operations summary (sent to officials every Monday)
- Monthly sustainability report (SDG 13 metrics)
- Quarterly performance reviews (ward comparisons)
- Annual climate resilience assessment [1][2][54]

FR-RAG4-09: The agent SHALL provide actionable recommendations:

- "Increase waste collection frequency in Ward 12 (demand increased 30% this month)"
- "Deploy additional air quality monitors in high-complaint zones"
- "Prioritize drainage maintenance before monsoon season (historical flooding risk)" [1][54]

FR-RAG4-10: The agent SHALL support custom data export:

- CSV, Excel, JSON formats

- Filtered datasets based on user criteria
- Aggregated or raw data options [1][54]

Non-Functional Requirements

NFR-RAG4-01: Query Response Time: Return analysis results within 10 seconds for 90th percentile queries [1][4].

NFR-RAG4-02: Data Freshness: Analytics based on data no older than 1 hour (except for monthly satellite data) [1].

NFR-RAG4-03: Forecast Accuracy: Achieve MAPE (Mean Absolute Percentage Error) < 15% for 7-day complaint volume forecasts [1][54].

NFR-RAG4-04: Scalability: Support queries across 5+ years of historical data (100M+ records) without performance degradation [1][10].

NFR-RAG4-05: Explainability: Provide data sources, calculation methods, and statistical significance for all insights [1][48][54].

NFR-RAG4-06: Visualization Quality: Generate publication-ready charts with proper labeling, legends, and color schemes [1].

NFR-RAG4-07: Multi-User Support: Handle 100 concurrent analysts running complex queries [1][10].

Technology Stack

- **LLM:** GPT-4o for complex reasoning + data-to-text generation [1][54]
- **Query Engine:** Langchain SQL Agent for natural language to SQL translation [1][47]
- **Data Warehouse:** PostgreSQL with TimescaleDB extension for time-series [1]
- **Analytics:** Pandas, NumPy, SciPy for statistical analysis [1]
- **ML Models:** XGBoost, TensorFlow/PyTorch for predictive models [1]
- **Visualization:** Plotly for interactive charts, Matplotlib for static reports [1]
- **Caching:** Redis for query result caching [1]
- **Orchestration:** Apache Airflow for scheduled report generation [1]

Input/Output Specifications

Input Schema:

```
{
  "query_id": "string (UUID)",
  "user_id": "string",
  "natural_language_query": "string",
  "filters": {
    "time_range": {
      "start_date": "ISO 8601",
      "end_date": "ISO 8601"
    },
    "spatial_filter": {
      "wards": ["array of ward IDs"],
      "zones": ["array of zone IDs"]
    },
    "category_filter": ["array of complaint categories"],
    "demographic_filter": "string (optional)"
  },
  "output_format": "string (chart/table/report/csv)",
  "visualization_preferences": {
    "chart_type": "string (line/bar/heatmap/scatter)",
    "color_scheme": "string"
  }
}
```

Output Schema:

```

{
  "query_id": "string",
  "query_interpretation": "string (LLM's understanding of query)",
  "data_summary": {
    "records_analyzed": "int",
    "time_range_covered": "string",
    "wards_included": ["array"]
  },
  "statistical_results": {
    "descriptive_stats": {
      "mean": "float",
      "median": "float",
      "std_dev": "float",
      "percentiles": {"25th": "float", "75th": "float"}
    },
    "trends": {
      "direction": "string (increasing/decreasing/stable)",
      "rate_of_change": "float",
      "statistical_significance": "float (p-value)"
    }
  },
  "insights": [
    {
      "insight_type": "string (trend/anomaly/correlation)",
      "description": "string",
      "severity": "string (low/medium/high)",
      "supporting_data": {"key": "value"}
    }
  ],
  "predictions": {
    "forecast_values": ["array of floats"],
    "confidence_intervals": [
      {"lower": "float", "upper": "float"}
    ],
    "forecast_horizon": "string"
  },
  "recommendations": [
    {
      "recommendation": "string",
      "rationale": "string",
      "expected_impact": "string",
      "priority": "string (low/medium/high)"
    }
  ],
  "visualizations": [
    {
      "visualization_id": "string",
      "type": "string (chart/map/table)",
      "url": "string (link to generated image/interactive chart)",
      "caption": "string"
    }
  ],
  "data_export_url": "string (optional, link to CSV/Excel download)",
  "metadata": {
    "processing_time_ms": "int",
    "data_sources": ["array of KB sources used"],
    "models_used": ["array of ML models applied"]
  }
}

```

Cross-Agent Coordination & Integration

Orchestration Pattern

The SeaS multi-agent system follows a **hierarchical orchestration pattern** with a central coordinator agent managing workflows [1][64][69][70]:

```
User Request → Central Coordinator Agent →  
[Agent 1, Agent 2, Agent 3, Agent 4 in parallel/sequence] →  
Coordinator Aggregates Results → User Response
```

Coordinator Responsibilities:

- Route user requests to appropriate agent(s)
- Manage inter-agent communication and data passing
- Handle failures and retry logic
- Aggregate results from multiple agents
- Maintain conversation context and memory [1][64][69]

Inter-Agent Data Flow Examples

Workflow 1: Complaint Submission → Resolution → Analysis

1. **Agent 1** receives complaint → classifies + verifies → outputs verified complaint
2. **Agent 2** receives verified complaint → optimizes assignment → outputs assignment + route
3. Field worker completes task → **Agent 3** receives proof → verifies completion → outputs verification
4. **Agent 4** ingests all data → updates analytics → generates insights [1][64]

Workflow 2: Predictive Resource Planning

1. **Agent 4** predicts complaint volume for next week → passes to Agent 2
2. **Agent 2** uses forecast to pre-allocate resources and schedule preventive maintenance
3. **Agent 1** adjusts fraud detection thresholds if spike in complaints predicted (reduces false positives during high-volume periods) [1][63][72]

Workflow 3: Quality Improvement Loop

1. **Agent 3** flags low-quality work completion → passes to Agent 2
2. **Agent 2** adjusts worker performance scores and future assignments
3. **Agent 4** analyzes patterns of verification failures → identifies training needs → recommends policy changes [1][63][72]

Shared Infrastructure

All agents share common infrastructure components [1][64][68]:

Unified Vector Store:

- Single vector database (Pinecone/Qdrant) with namespaced collections:
 - `complaints_collection` (Agent 1 & 4)
 - `historical_assignments_collection` (Agent 2)
 - `proof_images_collection` (Agent 3)
 - `documents_collection` (Agent 4)
- Enables cross-agent semantic search and knowledge reuse [1][65][68]

Shared Data Warehouse:

- PostgreSQL with schemas for each domain:
 - `complaints` schema (Agents 1, 2, 3, 4)
 - `resources` schema (Agent 2)
 - `verifications` schema (Agent 3)

- analytics schema (Agent 4)
- Enforces data consistency and enables cross-schema joins [1]

Common LLM Layer:

- All agents access same LLM providers (GPT-4o, Claude, Groq) via unified API gateway
- Prompt templates stored centrally for version control [1][44]

Message Queue for Async Communication:

- Apache Kafka topics for agent-to-agent messaging:
 - complaint_verified topic (Agent 1 → Agent 2)
 - assignment_created topic (Agent 2 → Field Workers)
 - proof_submitted topic (Field Workers → Agent 3)
 - verification_completed topic (Agent 3 → Agent 4)
- Enables decoupled, scalable architecture [1][64]

Monitoring & Observability

Agent Performance Tracking:

- Track per-agent metrics:
 - Latency (p50, p95, p99)
 - Throughput (requests/second)
 - Error rates
 - Accuracy scores
- Visualize in Grafana dashboards [1][10]

LLM Cost Tracking:

- Monitor token usage per agent
- Calculate cost per query
- Implement cost controls (monthly budgets) [1]

Quality Monitoring:

- A/B test agent modifications
- Collect human feedback on agent outputs
- Trigger retraining when accuracy degrades [1]

Implementation Roadmap

Phase 1: Foundation (Months 1-2)

- Set up infrastructure (databases, vector stores, LLM APIs)
- Build Agent 1 (Classification & Verification) with basic fraud detection
- Integrate with existing complaint submission system
- Deploy to pilot ward (500 complaints/month) [1]

Phase 2: Resource Optimization (Months 3-4)

- Build Agent 2 (Resource Allocation) with route optimization
- Integrate with fleet management system
- Add Agent 3 (Proof Verification) for basic image comparison
- Expand to 5 wards (2,000 complaints/month) [1][63]

Phase 3: Analytics & Insights (Months 5-6)

- Build Agent 4 (Data Analysis) with KPI dashboards
- Implement predictive models (complaint forecasting)
- Create public-facing leaderboards
- Scale to city-wide deployment (10,000+ complaints/month) [1][54]

Phase 4: Optimization & Scale (Months 7-12)

- Implement reinforcement learning for Agent 2
- Add advanced fraud detection for Agents 1 & 3
- Fine-tune LLM models on domain data
- Expand to 10 cities nationally [1][63][72]

Conclusion

This four-agent RAG architecture provides SeaS with a **scalable, intelligent, and accountable** system for urban climate resilience management. Each agent specializes in a critical workflow stage while sharing knowledge and coordinating through a unified platform. The result is a comprehensive solution that aligns with SDG 13 objectives, empowers citizens, optimizes municipal operations, and drives data-informed policy decisions [1][2][44][64].

By leveraging cutting-edge RAG technology, multimodal AI, and collaborative agent architectures, SeaS positions itself as a next-generation civic platform ready to tackle the urgent challenges of climate change in urban India [1][52][83].

References

- [1] SeaS Project Presentation - PCCOE International Grand Challenge 2025
- [2] SDG 13: Climate Action - UN Sustainable Development Goals
- [44] AWS - What is Retrieval-Augmented Generation (RAG)?
- [45] Advanced Real-Time Fraud Detection Using RAG-Based LLMs (ArXiv)
- [48] RAG-Powered Fraud Detection in E-Commerce
- [52] [Akira.AI](#) - Building Multimodal RAG Systems
- [53] Prompting Guide - RAG for LLMs
- [54] [Arya.AI](#) - RAG in Finance: Top 10 Use Cases
- [63] DataGrid - Enhance Resource Allocation with AI Agents
- [64] HuggingFace - Multi-Agent RAG System
- [65] Paragon - Vector Databases vs Knowledge Graphs for RAG
- [68] Google Cloud - Vector Database Choices in Vertex AI RAG Engine
- [69] Moveworks - A Complete Guide to Agentic RAG
- [70] Anthropic - How We Built Our Multi-Agent Research System
- [72] Zilliz - How AI Agents Handle Dynamic Resource Allocation
- [83] Xenoss - Building Enterprise AI Knowledge Base with RAG
- [86] Astera - Building a Knowledge Base for RAG Applications